Namal University Mianwali

Course: Advanced Database Systems
Course Code: CSC-371

Assignment No. 01

“Project Proposal”

Anila Younas	NUM-BSCS-2023-05
Shehr Bano	NUM-BSCS-2023-11
Submitted To	Dr. Muzamil Ahmed
Date	18/03/2025

Table of Contents

1. Project Title

QuickBite: Distributed Real-Time Food Delivery and Order Management Platform. It is a Polyglot Database Architecture Addressing Consistency, Concurrency, and Scalability Challenges in Large-Scale Systems.

2. Problem Statement

The food delivery services such as Foodpanda and Uber Eats are associated with three different forms of data that require highly dissimilar database behaviour. The ACID guarantees of financial transactions are required. The GPS position of riders requires fast write speed. The data of restaurant catalog should have a flexible structure that may depend on the restaurant. Any of the three would be forced to one database technology and this would create bottlenecks in its performance during peak hours or would create data reliability problems under failure [1].

Weaknesses of the existing platforms are evident in back ends. Order state logic handled in application code is not resistant to concurrent requests and leads to the phantom order and duplicate charges. Preparation time estimates are stored in fixed values hence ETA predictions will be invalid during rush periods where the kitchen will be congested. Relational databases are exposed to write amplification which comes because of constant GPS writes. Distributed catalog information in services becomes out of time when a customer lags behind and displays customers outdated menus and wrong working hours [1][6].

They are not complaints that can be seen by the users but rather the immediate consequence of applying inappropriate database technology to each type of data. The solution involves a deliberate allocation of every category of data to the database that is best suited to it about consistency, throughput and structure requirements. This is referred to as polyglot persistence and is based on the distributed systems theory [5][6].

3. Project Description

QuickBite suggests the design and partial development of a database architecture of a food delivery platform. The system is configured with two storage layers i.e. relational layer where transactional data is stored and document layer where catalogues and analytics are stored. A component of optional cloud deployment is also outlined. This is not intended to be a full-fledged product, but a technically viable demonstration of polyglot database design to a problem in the real world.

The system is used by four categories of users: customers who visit menus and place orders, restaurant operators who handle orders and change menus, riders who get assignments and post GPS positioning, and administrators who track system usage.

The principal functional elements are:

1. Order Lifecycle Management: Orders pass through states, which include PLACED, CONFIRMED, PREPARING, PICKED_UP, and DELIVERED. Database transitions are implemented by means of PL/SQL stored procedures, rather than in application code.
2. Menu and Inventory: MongoDB has the feature of embedding documents meaning that each restaurant can have its own structure without schema migrations.
3. Rider Dispatch and Location Tracking: The Rider GPS positions are stored in a high write MongoDB collection with an expiry of old records and geospatial index of nearest-rider search.
4. Dynamic ETA Calculation: No longer using static estimates, an aggregate pipeline makes use of historical order data to compute average preparation time per restaurant and per hour of day.

5. Payment and Audit Records: All the payments will be written in Oracle on a transactional basis. A trigger stores all changes in order status to audit table in the case of disputes.
6. Cross-Database Synchronization: Oracle has an outbox table that keeps track of state changes in the same transaction. These events are then forced to MongoDB by a scheduled job, maintaining both databases in eventual consistency without a distributed transaction.

4. Complex Computing Problem Identification

SAGA framework, as the system of Seoul Accord Graduate Attributes, has nine characteristics of defining a Complex Computing Problem. The six are highly present in this project.

Conflicting Requirements: CP behaviour (consistency over availability) is needed in payment records and AP behaviour (availability over consistency) is needed in rider location ingestion. Such limitations cannot be used together in one database technology enforcing architecturally inconsistent choices [8].

Depth of Analysis Required: There is no blueprint of how to assemble a PL/SQL state machine, MongoDB transactional outbox synchronization and ETA pipeline based on MongoDB aggregations into a compelling failure-tolerant system. The border of the integration of the two databases needs original design reasoning.

Depth of Knowledge Required: The appropriate solution would also require Oracle PL/SQL internals, MongoDB replica set semantics, pipeline aggregate performance, geospatial indexing, transaction compensation patterns in the cloud, and only one of these areas would be adequate on its own [3][4][9].

Consequences: Monetary imbalance is due to a payment that was lost. Failure to make a state transition halts a restaurant. Checkout of an unavailable item can be performed with the help of an inventory lost update. In platform scale, even a high side failure rate creates thousands of actual operational damages per day.

Interdependence: Both databases have five sub-systems that are interdependent about the placement of orders. The failure of any single one of the layers - trigger logic, outbox synchronization, or aggregation pipeline - is a design error that spreads to these other layers as a cascading failure.

Requirement Identification: Several parameters such as TTL expiry time, ideal read preference with replica lag and outbox policy cannot be stipulated in advance and need to be established by empirical measurement during implementation.

5. Proposed Database Technologies

There are two database technologies employed, and each of them is selected based on a particular reason. A optional cloud component has been described as well.

Technology	Data Domain	Role and Justification
Oracle Database XE (Relational / OLTP)	Orders, Payments, Users, Order Items, Audit Logs, Outbox Events	Oracle supports row-level locking of concurrent writers and full ACID compliance and PL/SQL support in implementing the order state machine as a stored procedure. They are partitioned tables, composite indexes and role-based access control. This is an absolute layer that cannot be compromised at all, financial records cannot be eventual consistent. The version of Oracle applied is Oracle XE, which is entirely free of charge [2][3][7].

MongoDB 7.x (Document / NoSQL)	Restaurant Catalogs, Rider Location History, Order History, ETA Aggregation	MongoDB offers a document model which enables each restaurant to have its menu structure without schema migrations. It has a write throughput and TTL index that justifies continuous GPS ingestion. The aggregation line estimates moving ETA using historical data. Three node replicas set offers automatic failover and read preference configurations [4][5].
Azure Cosmos DB (Cloud — Optional)	Cloud deployment of the MongoDB layer (if time allows)	If time permits, the MongoDB layer will be migrated to an Azure Cosmos DB on MongoDB API using an Azure Student account. Cosmos DB also supports MongoDB queries and aggregation pipelines, and thus no modifications in code are required. This would prove practical cloud database deployment as opposed to conceptual deployment. This element is optional and will only be tried in case core implementation has been done.

Redis and Kafka are excluded. These two are typical of production food delivery systems but would broaden without corresponding depth to the main argument on the database design. The Oracle-MongoDB integration and the synchronization mechanism between the two are already enough to form a problem of a semester level distributed database.

6. System Architecture Design

The system has four tiers. The client level consists of the customer app, restaurant operator interface, and rider app all of which are in communication with the backend via a REST API. The application service layer contains four services, including the Order Service (writes to Oracle), the Catalog Service (reads/writes MongoDB), the Dispatch Service (queries MongoDB to find nearby riders), and the Analytics Service (runs aggregation pipelines in MongoDB). The database layer contains two nodes, Oracle XE locally running and three node MongoDB replica set. The transactional outbox pattern is employed in the cross-database synchronization - when Oracle writes are made, the corresponding events are written to an event row in the same transaction, and a scheduled job sends these events to MongoDB on a 30-second interval [9].

System Architecture Diagram

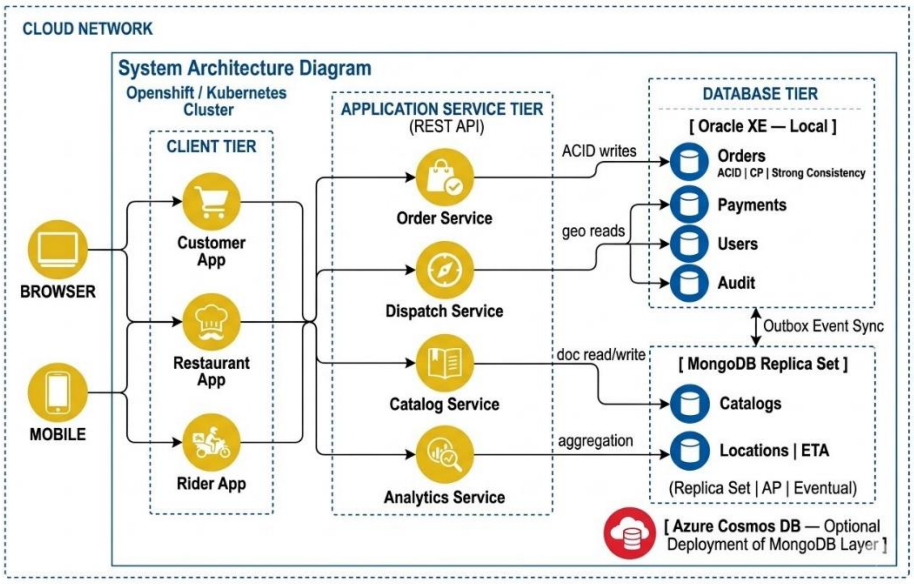

Figure 1: QuickBite System Architecture Diagram

7. Core Database Design

7.1 Oracle Schema

Table Name	Primary Key	Foreign Key(s)	Constraints & Partitioning Logic
USERS	User_ID	None	Stores hashed credentials; Role-based access (Customer, Rider, Admin).
RESTAURANTS	Rest_ID	None	Anchor for MongoDB documents; stores physical coordinates.
ORDERS	Order_ID	Cust_ID, Rest_ID	Range-Partitioned by Order_Date to optimize historical queries.
ORDER_ITEMS	Item_ID	Order_ID	Stores line-item details with NOT NULL constraints.
PAYMENTS	Pay_ID	Order_ID	ACID compliant; strictly linked to the transactional layer.
OUTBOX_EVENTS	Event_ID	Order_ID	Populated via AFTER UPDATE Trigger for cross-DB sync.

7.2 MongoDB Collections

1. Overview

Three collections are characterized. The restaurants collection stores the restaurants as single embedded documents with the complete menu, cuisine tags, opening hours, and the availability of items flags. Embedding is selected due to the fact that the bulk of the reads will fetch the entire document of a restaurant at a same time so no joins are required. The riderlocations collection contains GPS updates that include GeoJSON Point, timestamp and assignment status. Records that are more than 48 hours old are automatically

expired under TTL index. A 2dsphere index allows nearest-rider near-queries at dispatch time. Orderhistory collection contains a reflection of the finished records of the orders in the Oracle through outbox sync, and all analytical reads are offloaded to the Oracle primary [4].

2. Scaling and Horizontal Partitioning Strategy

To facilitate ingesting the high volume of data with high velocity on a platform-wide basis, the following plan is adopted:

Horizontal Scaling (Sharding): The riderlocations collection will be Sharded to avoid bottlenecks at a single node when the load is at its highest.

Selection of Shard key: The shard key is selected as cityzone.

Justification: This will guarantee that updates made by different geographic regions are committed by different shards and thus will still be able to achieve massive parallel write throughput without interfering with the primary Oracle instance. [4][5].

3. Key Design Decisions

1. The filtered restaurant browse query is supported by a compound index on (city_zone, cuisine_tags, is_active).
2. The ETA aggregation pipeline collects order history in groups by restaurant_id and hour of day, and calculates the average preparation_duration_minutes with $group and $avg stages. Findings are accumulated in an eta_estimates set on a refresh cycle [4].
3. Write concern will be majority to assign write riders to ensure that an assignment is not lost during a primary failover.

Read preference is secondaryPreferred for catalog and ETA reads, distributing read load across replica secondaries without affecting write path

4. Schema Visualization

The **Embedded Document** pattern of the restaurants collection as shown in the following JSON representation minimizes read latency, and also means that the complex joins are not necessary:

JSON

```
{
  "restaurant_id": 101,
  "name": "Namal Cafe",
  "cuisine": ["Pakistani", "Fast Food"],
  "menu": [
    { "item": "Biryani", "price": 350, "available": true },
    { "item": "Chai", "price": 60, "available": true }
  ],
  "location": { "type": "Point", "coordinates": [71.8, 32.6] }
}
```

8. Expected Challenges

Challenge	Description

Cross-DB Sync Failure	The outbox job might run out of synchronization either in the middle of a batch or lag behind in a busy system. Recovery should not duplicate or omit MongoDB records on retries.
Concurrent Inventory	A series of customers ordering the final item will generate Oracle row-level locking. Idempotent retry strategy should be used to prevent duplicate orders in transactions that are timed out.
PL/SQL Edge Cases	The state machine should support cancellations during delivery, restaurants indicating their orders are ready and waiting before they are assigned to a rider, timeouts caused by the scheduler. A different test case is required for each.
Pipeline Performance	The ETA aggregation pipeline can slug as orderhistory is increased. The use of indexes should be verified with explain with realistic sizable data.
Schema Evolution	To add a new field to the existing MongoDB documents during a project we have to perform an updateMany migration and set the field then use it in queries.
Consistency Management	Install Idempotent Consumers in the MongoDB sync job so that when a network failure happens, the same event of the Order Delivered is not recorded twice and therefore data is not duplicated.

9. Expected Outcomes

This project is expected to produce the following results at the end of the semester. Each is related directly to a particular point of the problem statement or CCP features described in Section 4.

1. The entire Oracle schema with six tables, range partitioning on ORDERS, composite keying and role-based access control.
2. An operational PL/SQL ORDERSTATEMACHINE procedure, where all invalid transitions are rejected, and outbox event AFTER UPDATE triggers and audit logging.
3. Three node replica set with all three collections in it set up in MongoDB, including TTL and 2dsphere indices on rider locations, compound index on restaurants, and majority write concern on assignments.
4. An ETA aggregation pipeline that generates an average preparation time per restaurant/hour, which is confirmed with explain output.
5. An effective outbox synchronization system to mid-batch failure and primary failover conditions.
6. Performance comparison between Oracle and MongoDB of read latency of a similar query of order history, which supports the polyglot design choice.
7. A demonstration of the CAP theorem observability - MongoDB secondaries provide stale data during a simulated partition, whereas Oracle rejects write to maintain consistency.
8. (Optional) Implementation of MongoDB layer to Azure Cosmos DB in case of time spare, which will showcase the deployment to cloud.

9. An ultimate report explaining what design choice was related to a particular issue that had been detected in the current food delivery systems.

References

[1] Elmasri, R. & Navathe, S. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

[2] Kyte, T. (2010). Expert Oracle Database Architecture (2nd ed.). Apress.

[3] Feuerstein, S. (2014). Oracle PL/SQL Programming (6th ed.). O'Reilly Media.

[4] Chodorow, K. (2019). MongoDB: The Definitive Guide (3rd ed.). O'Reilly Media.

[5] Sadalage, P. J. & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.

[6] Coulouris, G., Dollimore, J., Kindberg, T. & Blair, G. (2011). Distributed Systems: Concepts and Design (5th ed.). Addison-Wesley.

[7] Oracle Corporation. (2021). Oracle Database Concepts 21c. Oracle Technical Documentation.

[8] Brewer, E. A. (2000). Towards Robust Distributed Systems. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing, Portland, OR.

[9] Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning Publications.